

Multi-Source Candidate Data Transformer

Technical Design · Rikesh Yadav · rikeshyadav2780@gmail.com

Pipeline

`detect` → `extract` → `normalize` → `merge` → `confidence` → `project` → `validate`

Detect classifies each input (file extension / URL pattern) into a source type. **Extract** parses each source's native format into a common intermediate shape — one function per source type, each wrapped to never raise, so a broken source degrades to an empty record instead of crashing the run. **Normalize** converts phones to E.164, dates to YYYY-MM, and skills to canonical names via an alias table. **Merge** clusters intermediate records into one record per real candidate and resolves conflicts. **Confidence** scores each field. **Project** reshapes the canonical record per runtime config. **Validate** checks the projected output against the requested schema before returning it. These are split into independently-testable modules, and the canonical record is never mutated by the projection layer.

Canonical Schema & Normalized Formats

Schema follows the assignment's default table exactly (`candidate_id`, `full_name`, `emails[]`, `phones[]`, `location`, `links`, `headline`, `years_experience`, `skills[]`, `experience[]`, `education[]`, `provenance[]`, `overall_confidence`). **Phones** → E.164, via a self-contained heuristic normalizer (regex + configurable default-country fallback) rather than an external dependency, keeping the tool installable with zero friction. **Dates** → YYYY-MM, parsed from a dozen common formats, with "present" preserved literally for ongoing roles. **Skills** → canonical display names via an alias map ("js" → "JavaScript", "RL" → "Reinforcement Learning"), with unrecognized skills passed through rather than dropped. **Country** → ISO-3166 alpha-2 via lookup ("Taiwan" → "TW").

Merge & Conflict-Resolution Policy

Match key: two records are the same candidate if they share a normalized email or phone — name is deliberately not used as a match key (false-positive risk with common names; a documented trade-off, not an oversight). **Trust weights** resolve scalar conflicts (`full_name`, `headline`, `current role`): resume 0.9 > ats_json 0.8 > recruiter_csv 0.75 > linkedin 0.7 > github 0.6 > recruiter_notes 0.5 — highest-trust non-null value wins. **List fields** (`emails`, `phones`, `skills`, `education`) are unioned and deduplicated, never silently overwritten. **Confidence** starts at the winning source's trust weight, +0.15 (capped at 1.0) if 2+ independent sources agree on the same normalized value; `overall_confidence` averages all populated field confidences — directly encoding "honestly-empty beats wrong-but-confident."

Runtime Config / Projection

The internal CanonicalRecord is always full-fidelity. A separate projector reshapes it per a runtime JSON config: select a field subset, rename via `from` (supporting paths like `emails[0]` and list-projections like `skills[].name`), apply per-field normalize (E.164 / canonical), toggle confidence/provenance, and choose on_missing behavior (null / omit / error). A validation pass then checks the projected output's shape against the config before it's returned.

Edge Cases Handled

- Same candidate, conflicting data across 3 sources (different email/phone formats, job titles) — resolved via match-key clustering + trust-weighted selection; covered by automated tests.
- Malformed JSON / empty files — caught at the extractor level, source contributes zero records, run continues without crashing.
- Field-name mismatch between source and canonical schema (ATS sample uses `fullName/mobile/org/addr.nation`) — the extractor is exactly this remapping layer.
- Non-ISO country names ("Taiwan" instead of "TW") — converted via lookup at extraction time.
- Live external API failure (GitHub source: no network, rate limit, bad username) — defensive try/except degrades to empty list rather than crashing.

Deliberately Descoped (under the 36-hour window)

LinkedIn and resume (PDF/DOCX) extraction are not implemented — the extractor interface (`extract(source_type, path) -> list[dict]`) is built so either is a drop-in addition, not a redesign, but was judged lower-value than hardening the merge/confidence/projection core given the time constraint. Phone normalization uses a lightweight heuristic, not the full `libphonenumber` library, trading perfect international coverage for zero install dependencies. Scale testing at "thousands of candidates" was not performed; the design (no global state, single linear pass, dict-based clustering) should scale reasonably but wasn't load-tested.